

A close-up photograph of a young woman with long, wavy blonde hair. She is smiling and looking off to the right. She is wearing a blue and white horizontally striped sweater. The background is blurred, showing what appears to be an outdoor setting with a brick wall and some greenery.

Klasser som brugerdefinerede typer

Program:

- Hvad gør new?
- Hvordan laver vi en brugerdefineret type?
- Hvad er en konstruktør?
- Hvordan bruger vi vores nye type?
- `toString()`
- `equals()`
- Hvad er et værdiobjekt?
- Øvelser
- Opsummering

Vi har brugt Javas indbyggede typer, fx **String** til at data, fx

```
String givenName = "Victor";  
String familyName = "Lukic";  
String email = "lucky@victory.dk";
```

Det kan være *upraktisk* - og næsten *ulæseligt*

```
generateEmail(fromGivenName, fromFamilyName, fromEmail,  
              toGivenName, toFamilyName, toEmail,  
              subject, body);
```

Det kan være *mere praktisk* at samle relaterede data i én variabel

Hvad nu hvis vi vil gemme alle kontaktoplysninger for en person i én variabel?

```
Contact victor = new Contact(...);  
Contact thorkild = new Contact(...);  
  
generateEmail(victor, thorkild, subject, body);
```

Vi kan lave vores **egen type** ved at definere en ny klasse

Vi laver en ny fil **Contact.java**, en klasse **Contact** med tre felter

```
public class Contact {  
    String givenName;  
    String familyName;  
    String email;  
}
```


Contact-klassen er en slags **skabelon** for hvordan et **Contact**-objekt ser ud, dvs. *fornavn* (**givenName**), *efternavn* (**familyName**) og *email-adresse* (**email**)

Nu kan vi oprette instanser af **Contact** med **new**-nøgleordet

```
Contact victor = new Contact();
victor.givenName = "Victor";
victor.familyName = "Lukic";
victor.email = "lucky@victory.dk";

Contact thorkild = new Contact();
thorkild.givenName = "Thorkild";
thorkild.familyName = "Hansen";
thorkild.email = "thorkild@hansen.dk";

generateEmail(victor, thorkild, "Hej", "Hvordan går det?");
```

Hvad gør new?

- new opretter et nyt objekt i hukommelsen
- new returnerer en reference (adresse) til objektet

Referencen kan vi gemme i en variabel af typen **Contact**, fx

```
Contact victor = new Contact();
```

dvs.

- **victor** er en variabel af typen **Contact**
- **victor** indeholder en reference til et **Contact**-objekt

Vi kan bruge **victor** til at få adgang til objektets felter, fx

```
victor.givenName = "Victor";
```

new kalder også klassens konstruktør-metode

Hvad er en konstruktør?

En speciel metode, der

- har samme navn som klassen
- ikke har nogen returtype (ikke engang `void`)
- ikke har `static` i signaturen
- kaldes automatisk når vi laver et nyt objekt med `new`

Hvis vi *ikke* selv skriver en konstruktør, så laver Java en tom en for os

```
public class Contact {  
    String givenName;  
    String familyName;  
    String email;  
}
```


... er det samme som

```
public class Contact {  
    String givenName;  
    String familyName;  
    String email;  
  
    public Contact() {  
        // tom konstruktør  
    }  
}
```

Vi kan også selv skrive en konstruktør, fx

```
public class Contact {  
    String givenName;  
    String familyName;  
    String email;  
  
    public Contact(String gn, String fn, String pn) {  
        givenName = gn;  
        familyName = fn;  
        email = pn;  
    }  
}
```

Nu kan vi oprette et **Contact**-objekt med initialværdier

```
Contact victor = new Contact("Victor", "Lukic", "lucky@victory.dk");
```

Hvorfor skrev du ikke bare ...

```
public class Contact {  
    String givenName;  
    String familyName;  
    String email;  
    public Contact(String givenName, String familyName, String email) {  
        givenName = givenName;  
        familyName = familyName;  
        email = email;  
    }  
}
```

```
public class Contact {  
    String givenName;  
    String familyName;  
    String email;  
    public Contact(String givenName, String familyName, String email) {  
        givenName = givenName;  
        familyName = familyName;  
        email = email;  
    }  
}
```

Fordi parameterne (fx **givenName**) "skygger" for felterne med samme navn.

Java ved ikke om du mener feltet eller parameteren.

Løsning: Brug **this** til at referere til objektets felter

```
public class Contact {  
    String givenName;  
    String familyName;  
    String email;  
  
    public Contact(String givenName, String familyName, String email) {  
        this.givenName = givenName;  
        this.familyName = familyName;  
        this.email = email;  
    }  
}
```

Overloading af konstruktører

Kan i huske, da vi talte om **overloading** af metoder? En konstruktør er *bare* en metode, så ...

Som andre metoder, kan vi **overload** konstruktører

```
public class Contact {  
    String givenName;  
    String familyName;  
    String email;  
  
    public Contact(String givenName, String familyName, String email) {  
        this.givenName = givenName;  
        this.familyName = familyName;  
        this.email = email;  
    }  
  
    public Contact(String givenName, String familyName) {  
        this.givenName = givenName;  
        this.familyName = familyName;  
        this.email = "N/A";  
    }  
}
```

Spørgsmål: Hvor mange konstruktører har klassen **Contact** nu?

```
public class Contact {  
    String givenName;  
    String familyName;  
    String email;  
  
    public Contact(String givenName, String familyName, String email) {  
        this.givenName = givenName;  
        this.familyName = familyName;  
        this.email = email;  
    }  
  
    public Contact(String givenName, String familyName) {  
        this.givenName = givenName;  
        this.familyName = familyName;  
        this.email = "N/A";  
    }  
}
```

Nu kan vi oprette **Contact**-objekter med eller uden email

```
Contact victor = new Contact("Victor", "Lukic", "lucky@victory.dk");  
Contact thorkild = new Contact("Thorkild", "Hansen");
```

Men ikke uden parametre

```
Contact ukendt = new Contact(); // Fejl!
```

DEMO: Konstruktører



toString()

Vi er vant til at

```
int age = 22;  
System.out.println(age); // => "22"
```

men hvad sker der hvis vi prøver at printe et objekt?

```
Contact victor = new Contact("Victor", "Lukic", "lucky@victory.dk");  
System.out.println(victor); // => "Contact@6bc7c054"
```



```
System.out.println(victor); // => "Contact@6bc7c054"
```

Når vi printer et objekt, fx med `System.out.println()`, kaldes objektets `toString()`-metode

Hvis vi *har* lavet en `toString()`-metode i vores klasse, så bliver den kaldt

Har vi ikke lavet en `toString()`-metode, får vi en standard `toString()` og den returnerer altså bare

```
System.out.println(victor); // => "Contact@6bc7c054"
```

Hvorfor viser `toString()` ikke bare alle felterne i objektet?

Fordi Java ikke ved

- hvilke felter skal den vise?
- er nogle felter fx "hemmelige"?
- i hvilken rækkefølge?
- hvordan skal det formateres?

Løsning: Vi kan selv lave en `toString()`-metode i vores klasse

```
public String toString() {  
    return String.format("%s %s, tlf: %s",  
        givenName, familyName, email);  
}
```

```
Contact victor = new Contact("Victor", "Lukic", "lucky@victory.dk");  
System.out.println(victor); // => "Victor Lukic, tlf: lucky@victory.dk"
```

DEMO: toString()

The background is a blurred photograph of a crowd, likely at a public event or conference. On the right side, the back of a person's head with long, wavy red hair is visible, and a portion of their green eye is seen looking towards the left. The overall scene is out of focus, emphasizing the text in the foreground.

Immutable objects


```
Contact victor = new Contact("Victor", "Lukic", "lucky@victory.dk");  
victor.email = "lol@megalol.com"; // Åndrer victors email
```

Hvis vi *ikke* vil tillade at felterne i et objekt kan ændres efter oprettelse,
kan vi gøre dem **final**

```
public class Contact {  
    final String givenName;  
    final String familyName;  
    final String email;  
  
    public Contact(String givenName, String familyName, String email) {  
        this.givenName = givenName;  
        this.familyName = familyName;  
        this.email = email;  
    }  
}
```

Nu kan vi *ikke* ændre felterne efter oprettelse

```
Contact victor = new Contact("Victor", "Lukic", "lucky@victory.dk");  
victor.email = "lol@megalol.com"; // Fejl!
```

Objekter med **final** felter kaldes **immutable objects** (uforanderlige objekter)

String-klassen er et eksempel på en *immutable* klasse

```
String name = "Thorkild";  
String name2 = name.toUpperCase();  
System.out.println(name); // => "Thorkild"  
System.out.println(name2); // => "THORKILD"
```

dvs. `toUpperCase()` ændrer ikke i `name`, men returnerer en ny `String`.

DEMO: final filter

Værdiobjekter



Vi har kredset om begrebet **værdiobjekter** (value objects)
uden at definere det præcist

Værdiobjekter (value objects) er objekter, der

- kun indeholder data (felter) - ingen adfærd (metoder)
- ved sammenligning, sammenlignes deres felter og ikke deres referencer
- ofte er immutable (felterne er `final`)

Hvorfor bruge værdiobjekter?

- Samler relaterede data i én variabel
- Gør det nemmere at sende data rundt i programmet (fx som parametre til metoder)

Er **Contact** et værdiobjekt?

```
Contact victor1 = new Contact("Victor", "Lukic", "lucky@victory.dk");  
Contact victor2 = new Contact("Victor", "Lukic", "lucky@victory.dk");  
  
System.out.println(victor1 == victor2); // => false
```

Spørgsmål: Burde **victor1 == victor2**?

`vector1 == vector2` er for reference-typer det samme som at kalde
`vector1.equals(vector2)`

Standard for reference-typer: `equals()` sammenligner referencer (adresser), ikke felter.

```
Contact victor1 = new Contact("Victor", "Lukic", "lucky@victory.dk");  
Contact victor2 = victor1;  
System.out.println(victor1 == victor2); // => true
```

Så skal **Contact** være et værdiobjekt, skal vi overskrive **equals()**-metoden

```
public boolean equals(Object other) {  
    // ...  
}
```

For **Contacts** kunne det (simplificeret) se således ud

```
public boolean equals(Object other) {  
    Contact contact = (Contact) other;  
    return givenName == contact.givenName &&  
        familyName == contact.familyName &&  
        email == contact.email;  
}
```



```
public class Contact {  
    final String givenName;  
    final String familyName;  
    final String email;  
  
    public Contact(String givenName, String familyName, String email) {  
        this.givenName = givenName;  
        this.familyName = familyName;  
        this.email = email;  
    }  
  
    public boolean equals(Object other) {  
        Contact contact = (Contact) other;  
        return givenName.equals(contact.givenName) &&  
            familyName.equals(contact.familyName) &&  
            email.equals(contact.email);  
    }  
}
```

DEMO: equals()

Nu sammenlignes felterne i stedet for referencerne

```
Contact victor1 = new Contact("Victor", "Lukic", "lucky@victory.dk");  
Contact victor2 = new Contact("Victor", "Lukic", "lucky@victory.dk");  
  
System.out.println(victor1 == victor2); // => true
```

Er der ikke nogen nemmere måde at lave værdiobjekter på?

Jo, vi kan bruge **records**

```
public record Contact(String givenName, String familyName, String email) {
```

```
    Contact victor1 = new Contact("Victor", "Lukic", "lucky@victory.dk");  
    Contact victor2 = new Contact("Victor", "Lukic", "lucky@victory.dk");
```

```
    System.out.println(victor1 == victor2); // => true
```

Done!

Så værdiobjekter ... records

DEMO: records



Nævn tre ting du tager med fra i dag?